Tema 3

Lenguajes para el almacenamiento y transmisión de información

Lenguaje de marcas y sistemas de gestión de la información

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 4
2. Tipos de lenguajes 5
2.1 XML 5
2.2 JSON 6
3. XML: Estructura y sintaxis 10
3.1 Estructura general 10
3.2 Visualización 12
3.3 Elementos: prólogo y ejemplar 13
3.4 Sintaxis 14
3.5 Secciones CDATA 16
3.6 Definición de Tipo de Documento (DTD) 16
4. Etiquetas 19
4.1 Reglas etiquetas 19
4.2 Nombres XML 20
5. Herramientas de edición 22
5.1 XMLSpy 22
5.2 Oxygen 22
5.3 XML Copy Editor 23
6. Elaboración de documentos bien formados 24
6.1 Definición de tipo de documento (DTD, Document Type Definition) 26
6.1.1 Inconvenientes de DTD 26
6.1.2 Declaración 26
6.1.3 Componentes 27
6.2 Esquema XML (XSD, XML Schema Definition) 30
6.2.1 Asociación con documentos XML 31
6.2.2 Elementos de lenguaje 32
6.2.3 Tipos de datos XSD 33
6.2.4 Facetas de los tipos de datos 35

7. Utilización de espacios de nombre en XML 39
8. Resumen 43
Glosario 44
Bibliografía 46

Objetivos de la unidad

- Identificación y clasificación de los tipos de lenguajes para el almacenamiento y transmisión de información.
- Análisis de la estructura de un documento XML e identificación de su sintaxis.
- Reconocer y diferenciar las etiquetas empleadas en XML.
- Manejar herramientas de edición y validación de documentos XML.
- Elaborar documentos XML bien formados y válidos mediante el uso de DTD y XML Schema.
- Aplicar correctamente los espacios de nombres en documentos XML.

1. Introducción

Esta unidad se enfoca en los lenguajes utilizados para almacenar y transmitir información, en particular, los **lenguajes XML y JSON**. Se detallan los diferentes tipos de lenguajes y se profundiza en las características y usos de cada uno de ellos.

A continuación, se analizará de forma detallada el lenguaje XML, explicando su estructura y sintaxis, junto con las reglas básicas que debe cumplir, los elementos y las etiquetas que lo componen. También se presentan las herramientas más utilizadas para la edición de documentos XML, como XMLSpy, Oxygen y XML Copy Editor.

Finalmente, se abordará la elaboración de documentos bien formados y válidos, introduciendo la definición de tipo de documento (DTD) y el esquema XML (XSD), así como sus componentes y tipos de datos. Además, se explicará el uso de los espacios de nombres en XML, fundamentales para la correcta organización y validación de documentos.

2. Tipos de lenguajes

Los lenguajes usados para almacenamiento y transmisión de información pueden ser de dos tipos:

- De marcas: XML (eXtended Markup Language).
- De listas: JSON (JavaScript Object Notation).

2.1 XML

Definición

Se trata de un estándar que se creó con el objetivo principal de poder **adecuar el uso del anterior lenguaje SGML a la web.**

Una de las claves para el éxito de este formato es que su estándar **se basa en texto**, con el objetivo de ofrecer una representación adecuada de todo tipo de información estructurada, por ejemplo, en datos o documentos.

En XML, todas las etiquetas deben cerrarse correctamente, lo que ayuda a reducir errores de sintaxis. Además, las herramientas que procesan este tipo de archivos detectan y notifican cualquier problema en la estructura.

XML se basa en el uso de etiquetas, que pueden estar vacías, contener texto o incluir otros elementos. Dichas etiquetas siempre están delimitadas entre < > y deben cerrarse con </ >.

El estándar también permite el uso de atributos, los cuales aportan propiedades adicionales a los elementos, mejorando su funcionalidad y permitiendo añadir información complementaria.

```
<?xml version="1.0" encoding="UTF-8"?>
<Curso>
  <Nombre>Lenguajes de Marcas</Nombre>
  <Modulo>XML Básico</Modulo>
  <Profesor>Laura Sánchez</Profesor>
  <Horas>60</Horas>
</Curso>
```

En este ejemplo se representa un curso, con información básica como su nombre, el módulo, el profesor y el número de horas.

Entre las **ventajas** de este protocolo destacan su formato estructurado y mucho más fácil de comprender por la máquina.

El uso de **espacios de nombres en XML** puede facilitar la definición de estructuras complejas y promover la reutilización de código, lo cual lo hace muy útil en proyectos grandes o con múltiples colaboradores.

Sin embargo, también es importante señalar sus desventajas:

- Al ser un formato muy técnico, puede resultar complicado para las personas que no estén familiarizadas con él.
- Es un lenguaje estricto, lo que implica un mayor tiempo de procesamiento en comparación con otros formatos como JSON.
- Un error mínimo en la definición de espacios de nombres puede provocar que todo el documento se considere inválido.

Estas limitaciones han hecho que XML, aunque sigue siendo muy usado, haya perdido terreno en el intercambio de datos en favor de formatos más ligeros como JSON.

2.2 JSON

Definición

El **formato JSON** (JavaScript Object Notation) es una alternativa innovadora para el intercambio de información que ofrece una eficacia similar al formato XML. Sin embargo, JSON cuenta con ventajas adicionales, siendo la más destacada su capacidad para presentar la información de una manera más legible para las personas.

JSON se basa en un subconjunto específico del lenguaje de programación JavaScript y se ha convertido en una opción popular para intercambiar información entre diferentes aplicaciones y servicios web.

El **protocolo JSON** se apoya en dos estructuras fundamentales:

- **Una colección de pares nombre/valor.** Conocidos como objetos, diccionarios o registros en otros lenguajes.
- **Una lista ordenada de valores,** lo que en muchos lenguajes se denomina array, vector, lista.

Estructura básica

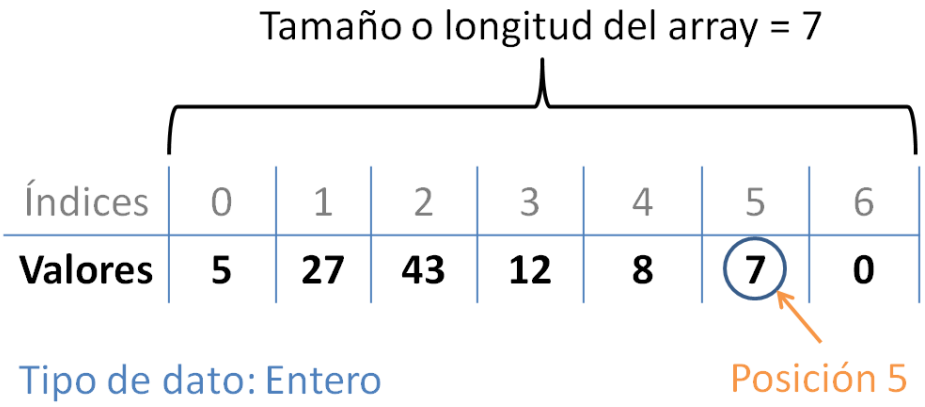
- Un objeto se define con llaves {} y está compuesto por pares clave: valor.
- Un array se define con corchetes [ ] y contiene valores separados por comas.

Ejemplo de código JSON:

```
{
  "curso": "Lenguajes de Marcas",
  "modulo": "Unidad 3 - Validación XML y JSON",
  "profesor": "Ana López",
  "alumnos": [
    { "nombre": "Carlos", "nota": 8.5 },
    { "nombre": "Lucía", "nota": 9.2 },
    { "nombre": "Miguel", "nota": 7.8 }
  ]
}
```

En este ejemplo:

- "curso", "modulo" y "profesor" son pares nombre/valor dentro de un objeto.
- "alumnos" contiene un array de objetos, donde cada objeto representa un estudiante con su nombre y nota.



Estructura de una array

Un valor en JSON puede ser:

- Una cadena de texto entre comillas dobles.
- Un número: Un **número** es como los números en Java. Puede tener parte entera + decimal; entera + exponente y entera + decimal + exponente.
- Booleanos: true o false.
- Null: para indicar ausencia de valor.
- Objeto: definido entre llaves { }.
- Array: definido entre corchetes [ ].

Una **cadena de texto** es una secuencia de cero o más caracteres Unicode, rodeados por dobles comillas. Se usa el carácter barra invertida (\) como marca de escape o de finalización.

En otros lenguajes, como Java, una cadena en JSON equivale al tipo String.

Ejemplo práctico en JSON

Se dispone de un array con dos objetos (cada objeto representa una ciudad). Cada ciudad tiene varios pares nombre/valor: nombre, población y una lista de monumentos.

```
[
 {
  "ciudad": "Roma",
  "poblacion": 2873000,
  "monumentos": ["Coliseo", "Panteón", "Fontana di Trevi"]
 },
 {
  "ciudad": "Kioto",
  "poblacion": 1475000,
  "monumentos": ["Kinkaku-ji", "Fushimi Inari", "Castillo Nijo"]
 }
]
```

Equivalencia en XML

El mismo ejemplo, representado en XML, tendría esta forma:

```
<ciudades>
 <ciudad>
  <nombre>Roma</nombre>
  <poblacion>2873000</poblacion>
  <monumentos>
   <monumento>Coliseo</monumento>
   <monumento>Panteón</monumento>
   <monumento>Fontana di Trevi</monumento>
  </monumentos>
 </ciudad>
 <ciudad>
  <nombre>Kioto</nombre>
  <poblacion>1475000</poblacion>
  <monumentos>
   <monumento>Kinkaku-ji</monumento>
   <monumento>Fushimi Inari</monumento>
   <monumento>Castillo Nijo</monumento>
  </monumentos>
 </ciudad>
</ciudades>
```

Sabías que

Existen herramientas que permiten convertir automáticamente entre JSON y XML, como XML-JSON Converter o XML to JSON. Estas utilidades son muy usadas en desarrollo web para adaptar datos entre distintos sistemas.

3. XML: Estructura y sintaxis

3.1 Estructura general

En un documento XML, la información se organiza de manera jerárquica, lo que significa que los elementos pueden contener otros elementos, estableciendo relaciones de padres, hijos, hermanos, ascendientes y descendientes.

Esta organización facilita la comprensión tanto por parte de las personas como de las máquinas, ya que cada nivel jerárquico refleja una relación lógica entre los datos.

A esta relación se le denomina árbol de nodos, ya que la estructura XML puede visualizarse como un árbol donde existe un elemento raíz que contiene al resto de elementos.

Ejemplo:

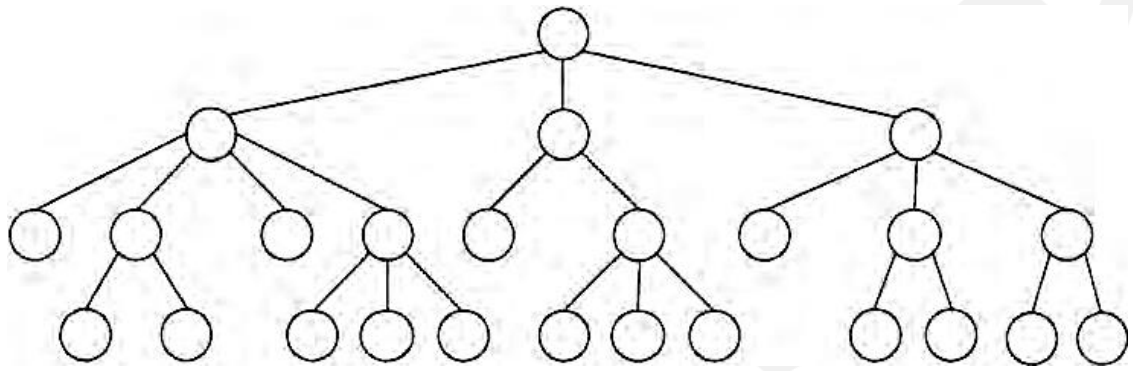
```
<universidad>
 <facultad>
  <nombre>Facultad de Ingeniería</nombre>
  <carreras>
   <carrera>
    <nombre>Informática</nombre>
    <duracion>4 años</duracion>
   </carrera>
   <carrera>
    <nombre>Civil</nombre>
    <duracion>5 años</duracion>
   </carrera>
  </carreras>
 </facultad>
</universidad>
```

En este ejemplo:

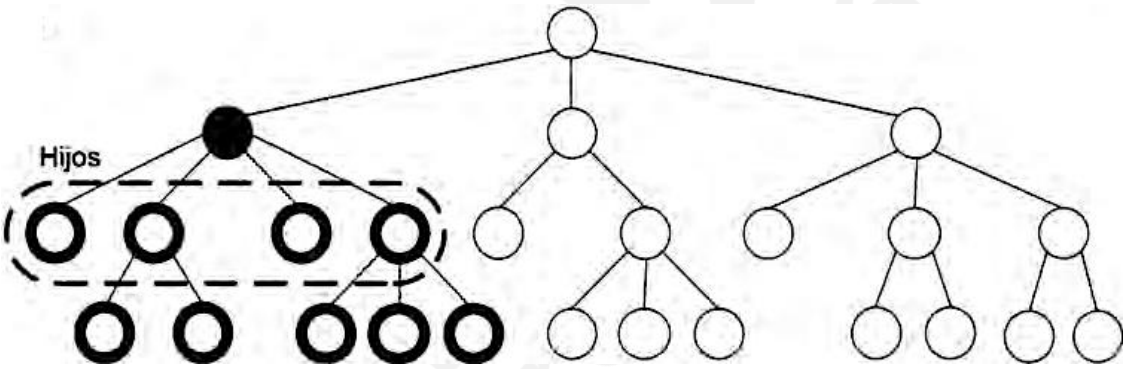
- <universidad> es el elemento raíz.
- <facultad> es un hijo del elemento raíz.
- <carreras> es descendiente de <facultad>.
- Los elementos <carrera> son hermanos entre sí.
- <nombre> y <duracion> son hijos de cada <carrera>.

De esta manera, los datos quedan perfectamente estructurados y relacionados, permitiendo una lectura clara y un procesamiento eficiente.

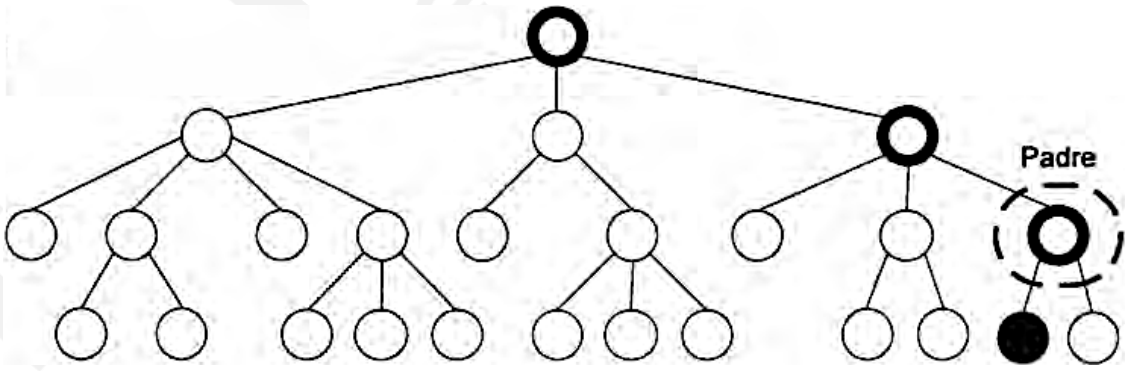
Original:



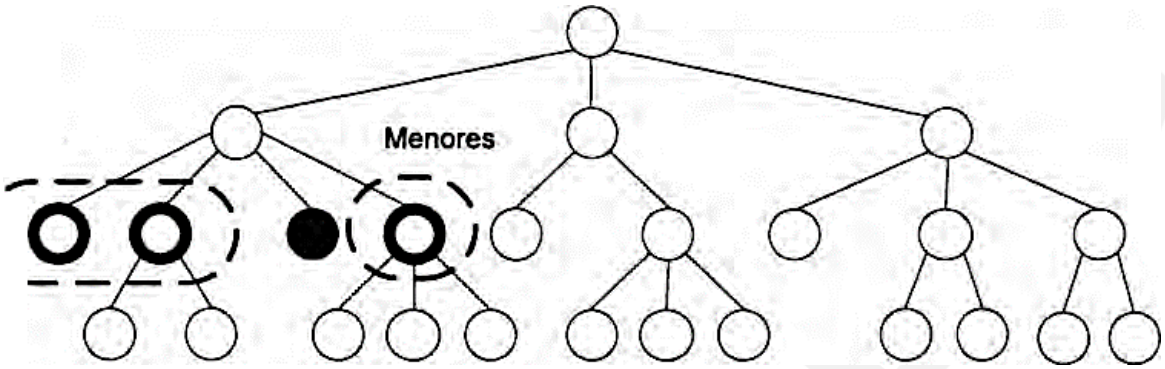
Descendientes:



Ascendentes:



Hermanos:



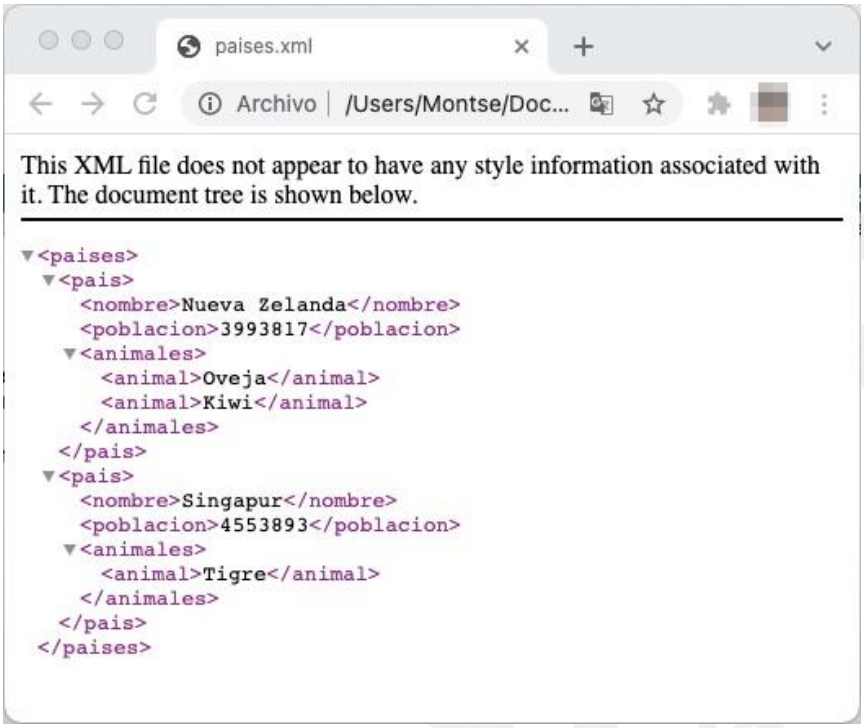
Fuente de las imágenes: https://es.slideshare.net/anuskaeg/lenguajes-de-marcas-xml

3.2 Visualización

Un documento no dispone de una visualización concreta en un navegador ya que su propósito principal no es definir la apariencia, sino estructurar y almacenar datos.

Cuando se abre un archivo XML en un navegador web, este simplemente muestra la jerarquía de etiquetas y el contenido en texto plano, sin aplicar estilos ni formatos visuales.

Para dar una representación visual atractiva o personalizada, es necesario combinar XML con tecnologías adicionales como CSS o transformaciones XSLT, que permiten aplicar formato o convertir la información a otros lenguajes (por ejemplo, HTML).



Visualización de un documento XML

3.3 Elementos: prólogo y ejemplar

Los documentos XML pueden estar formados por una parte opcional llamada **prólogo** y otra parte obligatoria llamada **ejemplar**:

- **Prólogo**: Acción inicial en un documento XML que proporciona al intérprete encargado de procesar el documento todos los datos necesarios para realizar su trabajo. Tiene dos partes:
  - **Definición de XML:** Indica la versión de XML que se está utilizando, la codificación empleada para representar los caracteres y la autonomía del documento. Hay tres opciones disponibles para la autonomía del documento: autónomo, independiente y dependiente.

```
<?xml versión= "1.0" ?>
<?xml versión= "1.0" encoding="iso-8859-1" ?>
<?xml versión= "1.0" encoding="iso-8859-1" standalone="yes" ?>
```

En el ejemplo dado, el documento se considera independiente, de lo contrario, el valor del atributo standalone sería "no".

  - Declaración del tipo de documento: comienza con la cadena: <!DOCTYPE Nombre_ejemplar ...> El nombre_ejemplar donde el

nombre_ejemplar debe ser igual al del documento XML en cuestión.

- **Ejemplar**: Es la parte más importante del documento, ya que contiene los datos reales y está formado por elementos anidados que definen la estructura del documento. Los elementos pueden contener otros elementos y/o atributos y el elemento raíz del documento XML es el ejemplar mismo. Si el documento XML es independiente, el atributo **standalone** tendrá el valor "yes"; de lo contrario, tendría el valor "no". Todos los demás elementos de un documento XML están contenidos en el mismo ejemplar.

Prologo
Cuerpo

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="no"?>
<! DOCTYPE persona SYSTEM "persona.dtd"
<persona>
    <nombre>Luis</nombre>
    <apellidos>Pérez</apellidos>
</persona>
```

Prólogo y ejemplar en un documento XML. Fuente: https://pt.slideshare.net/profesoryesith/xml-65711450

3.4 Sintaxis

Existen varias maneras para representar visualmente los datos de un documento XML:

- Una forma es **mediante una hoja de estilo CSS** que indique al navegador cómo convertir cada elemento del documento XML en un elemento visual. Se lograría con la siguiente instrucción de procesamiento que se verá más adelante:

```
<?xml-stylesheet type="text/css" href="estilos.css"?>
```

- Otra manera es mediante el uso de una **hoja de transformaciones XSLT**. La instrucción equivalente para asociar a un documento XML una hoja de transformaciones XSLT es:

```
<?xml-stylesheet type="text/xsl" href="transforma.xsl"?>
```

- También se podría hacer mediante el **uso de un lenguaje de programación**, como Java o JavaScript, que procese el documento XML y permiten manipularlo para generar diferentes salidas.

Cada documento XML tiene una estructura específica compuesta por diferentes tipos de **nodos o componentes**, entre ellos:

- **Raíz**: Que se representa como "/" y se ubica por encima de todos los elementos del documento XML.
- **Elementos**: Es la unidad básica de un documento XML. Son delimitadores/contenedores de información. Al igual que los elementos de HTML, se identifican por una etiqueta de apertura (como <persona>) y una de cierre (como </persona>). La información contenida en un elemento XML se ubica entre su etiqueta de apertura y su etiqueta de cierre. El contenido puede ser texto, otros elementos o estar vacío. Además, existen algunos tipos de **elementos especiales**, tales como:
  - Elemento raíz: Es obligatorio en todo documento XML bien formado, ya que es el contenedor principal de todos los demás elementos y no tiene ascendentes ni hermanos directos. También se conoce como elemento documento.
  - Elementos vacíos: Son elementos que no tienen contenido y se representan mediante una única etiqueta de apertura y cierre combinadas en una sola.
    - Ejemplo elemento sin contenido y sin atributos: <separador />
    - Ejemplo elemento sin contenido pero con atributos: <separador cantidad="7" />
- **Atributos**: Son como los atributos de HTML. Son pares nombre=valor que permiten especificar datos adicionales de un elemento. La asignación de valores a los atributos de un elemento en XML se realiza dentro de la etiqueta de apertura del mismo. Para ello, se utiliza el signo igual (=). Es importante destacar que, aunque los atributos puedan representar distintos tipos de datos, todos se tratarán como texto y se encerrarán entre comillas simples o dobles. Se usan fundamentalmente para almacenar información sobre la información (meta¬ información) contendida en el documento.

  Ejemplo: <distancia **unidades="km"**>70</distancia>

  También se emplean para recoger información identificativa del elemento que permita distinguirlo de otro elemento.

  Ejemplo: <persona **nif="12345678Z"**> ...</persona>
- **Texto**: Representa los datos en sí mismos. Aparece como contenido dentro de un elemento o como valor de un atributo, pero nunca fuera de ellos.

Estructura de un documento XML. Fuente: https://pt.slideshare.net/profesoryesith/xml-65711450

3.5 Secciones CDATA

En XML, cuando se quiere incluir contenido que no debe ser interpretado como etiquetas, se utiliza CDATA (Character Data). Todo lo que se coloque dentro de <![CDATA[...]]> será tratado como texto literal, sin procesarse.

Ejemplo:

Se quiere introducir dentro de un elemento <codigo> un fragmento de HTML para que el analizador XML no lo interprete como etiquetas, sino como texto:

```
<codigo>
  <! [CDATA [
     <html><body><h3>Título de la
página</h3></body></html>
  ]] >
</codigo>
```

De esta forma, el contenido se conserva tal cual sin ser procesado como parte de la estructura XML.

3.6 Definición de Tipo de Documento (DTD)

- **Definición de Tipo de Documento (DTD):** Permite definir reglas que fuercen ciertas restricciones sobre la estructura de un documento XML aunque su existencia no es obligatoria. Un documento XML que cumple con las reglas de sintaxis y estructura establecidas se considera bien formado. Si además se ajusta a las restricciones definidas en un documento de declaración de tipos, entonces se considera válido.

Aunque no es obligatorio definir una DTD, se recomienda incluirla en la segunda línea del documento XML, justo después de la declaración inicial <?xml ...?> y antes del elemento raíz.

Ejemplo de documento XML con DTD externa vinculada:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE elementos SYSTEM "elementos.dtd">
<!-- Comentario explicativo del documento -->
<?xml-stylesheet type="text/css" href="elementos.css"?>
<elementos>
  <elemento id="1">Ejemplo 1</elemento>
  <elemento id="2">Ejemplo 2</elemento>
</elementos>

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE elementos SYSTEM "elementos.dtd">
<!-- Comentario explicativo del documento -->
<?xml-stylesheet type="text/css" href="elementos.css"?>
<elementos>
  <elemento id="1">Ejemplo 1</elemento>
  <elemento id="2">Ejemplo 2</elemento>
</elementos>
```

Explicación del ejemplo:

1. Instrucción de procesamiento inicial que indica que el documento está en formato XML.
2. Inclusión de un DTD externo (archivo elementos.dtd).
3. Inserción de un comentario.
4. Instrucción que vincula una hoja de estilos (elementos.css).
5. Declaración del elemento raíz <elementos>.
6. Definición de elementos descendientes dentro del raíz.
7. Cierre del elemento raíz.

Más info

Para ampliar información sobre XML: Estructura y sintaxis visualiza el vídeo:

Texto – 3 – Tutorial XM básico en español

4. Etiquetas

Definición

Las etiquetas XML son elementos fundamentales en XML, ya que delimitan y definen los elementos que contienen información.

En general, las etiquetas XML se pueden **clasificar en tres tipos**:

- Etiqueta de inicio: Se encuentra al principio de cada elemento XML no vacío y se utiliza para marcar el inicio del elemento. Por ejemplo, <address> es una etiqueta de inicio.
- Etiqueta de cierre: Cada elemento que tiene una etiqueta de inicio debe finalizar con una etiqueta de cierre. Por ejemplo, </address> es una etiqueta de cierre. Cabe destacar que las etiquetas de cierre incluyen una barra ("/") antes del nombre del elemento.
- Etiqueta vacía: Se utiliza para representar un elemento sin contenido. Un elemento vacío puede ser representado de dos maneras diferentes:
  1. Mediante una etiqueta de inicio seguida inmediatamente por una etiqueta de cierre: <hr></hr>
  2. Mediante una etiqueta de elemento vacío: <hr /> Las etiquetas de elemento vacío se pueden utilizar para cualquier elemento que no tenga contenido.

4.1 Reglas etiquetas

Algunas **recomendaciones importantes** que se deben seguir al utilizar etiquetas XML son las siguientes:

- Las etiquetas XML **son sensibles a las mayúsculas y minúsculas**. Las etiquetas XML son *case sensitive* por lo tanto, se deben utilizar siempre las mismas mayúsculas y minúsculas en todas las apariciones de una etiqueta XML en el documento. Ejemplo incorrecto:

```
<Libro>Ejemplo</libro>
```

Ejemplo correcto:

```
<libro>Ejemplo</libro>
```

- Las etiquetas XML deben ser cerradas en el orden adecuado. Esto significa que una etiqueta abierta dentro de otra etiqueta debe ser cerrada antes de cerrar la etiqueta exterior. De lo contrario, el documento no será válido según la sintaxis de XML. Ejemplo incorrecto:

```
<Libro>Ejemplo</libro>
```

Ejemplo correcto:

```
<persona><nombre>Lucía</nombre></persona>
```

Es importante seguir estas normas al utilizar etiquetas XML para asegurar que los documentos sean válidos y se puedan procesar correctamente.

4.2 Nombres XML

En XML, tanto los elementos como los atributos deben cumplir ciertas reglas al momento de definir sus nombres válidos. Estas normas son estrictas y garantizan que los documentos se puedan interpretar de manera correcta por cualquier procesador XML.

Reglas principales:

- **Inicio válido:**

El nombre debe comenzar por una letra (incluyendo letras acentuadas o de alfabetos no latinos), un guion bajo _ o un dos puntos : (aunque este último se desaconseja porque suele usarse en espacios de nombres).

Correcto:

```
<nombre>Ejemplo</nombre>
```

Incorrecto:

```
<1dato>Ejemplo</1dato>
```

Caracteres permitidos después del inicio:

Se pueden incluir letras, dígitos, guiones bajos _, guiones medios -, comas , y dos puntos :.

Ejemplo válido:

```
<cliente_2025>Juan</cliente_2025>
```

Restricción con "XML":

Los nombres que comiencen con las letras XML (en mayúsculas, minúsculas o combinados) están reservados y no deben utilizarse, ya que están destinados a la estandarización.

Incorrecto:

```
<xmlDatos>Contenido</xmlDatos>
```

Prohibición de espacios y caracteres especiales:

Los nombres no pueden contener espacios en blanco ni símbolos como comillas ' ", signo de dólar $, acento circunflejo ^, porcentaje % o punto y coma ;.

Incorrecto:

```
<nombre cliente>Pedro</nombre cliente>
```

En resumen, cumplir estas reglas asegura la creación de documentos XML bien formados, evitando errores en la validación y en la interoperabilidad entre aplicaciones.

5. Herramientas de edición

Existen multitud de herramientas relacionadas con el manejo de **XML** y sus tecnologías asociadas (DTD, XSD, XPATH, XSLT, …). Algunas son muy sofisticadas y, en general, con licencias propietarias. Hay otras más sencillas, también de código abierto.

5.1 XMLSpy

El software **XMLSpy** de Altova es un editor muy conocido de XML y JSON que se utiliza para modelar, editar, transformar y depurar tecnologías relacionadas con XML. Este editor cuenta con una variedad de herramientas que permiten crear aplicaciones avanzadas, como un diseñador de esquemas gráficos, funciones de generación de código, conversores de archivos, depuradores y generadores de perfiles. XMLSpy puede trabajar con XSD, XSLT, XQuery, XBRL, JSON y otras tecnologías similares.

Más info

Para ampliar información sobre las funciones de XMLSpy y descargarlo puedes hacer clic en el siguiente enlace: https://www.altova.com/download-xml-editor-b?gclid=CjwKCAjw_MqgBhAGEiwAnYOAen0ADyCXJSTpvkrqO6vpV8Ks2KU9yY6gEEw3C6t5OezuVoNpObxvfBoCT0gQAvD_BwE

5.2 Oxygen

Oxygen XML Editor es una solución completa de edición XML para desarrolladores y autores de contenido. Ofrece una amplia variedad de herramientas para editar XML que cubren una gran cantidad de estándares y tecnologías relacionadas. Oxygen XML Editor incluye todas las características de Oxygen XML Developer y Oxygen XML Author.

Más info

Para ampliar información sobre las funciones de Oxygen y descargarlo puedes hacer clic en el siguiente enlace:

https://www.oxygenxml.com/

5.3 XML Copy Editor

XML Copy Editor es un editor de validación XML rápido, gratuito y de código abierto, que resulta ideal para quienes buscan una herramienta ligera pero funcional.

Más info

Para ampliar información sobre las funciones de XML Copy Editor y descargarlo puedes hacer clic en el siguiente enlace:

http://xml-copy-editor.sourceforge.net/

6. Elaboración de documentos bien formados

Un documento XML bien formado se refiere a aquel que sigue todas las reglas sintácticas definidas para XML. Si un documento no está bien formado, los procesadores XML lo rechazarán. Sin embargo, es importante no confundir la validez con la corrección de la sintaxis. Una definición precisa de un documento XML válido es que cumpla con las normas de formato y la definición de un lenguaje de marcado particular.

En otras palabras, todos los documentos válidos son bien formados, pero no todos los documentos bien formados son válidos.

Un procesador XML conforme con la Especificación XML comprobará que el documento cumple estas restricciones. Si no lo hace, el error será tratado como fatal: el procesador informará a la aplicación y detendrá su ejecución.

Fundamentalmente, el objetivo de estas restricciones es asegurar que los documentos XML puedan ser interpretados por los procesadores XML sin ninguna ambigüedad, de manera que todos los elementos (y atributos) quedan perfectamente definidos.

Para que un documento XML sea interpretado correctamente por los procesadores XML, es necesario que los elementos estén formados de acuerdo a ciertas reglas. Estas reglas son las siguientes:

- Todo documento XML debe tener un elemento raíz y solamente uno.
- Cada elemento debe tener una etiqueta de inicio y otra de cierre. En caso de que un elemento esté vacío, se puede utilizar una etiqueta de elemento vacío, la cual se construye reemplazando el carácter ">" por "/>" en la etiqueta de inicio.
- Al anidar elementos, es importante asegurarse de que ningún elemento se cierre antes de que todos los elementos contenidos dentro de él hayan sido cerrados.
- Al crear un documento XML, es importante tener en cuenta las reglas de sintaxis para las etiquetas de inicio y cierre de los elementos. Es obligatorio que el nombre de la etiqueta de cierre sea idéntico al nombre de la etiqueta de inicio, incluyendo mayúsculas y minúsculas.

  Los nombres de las etiquetas:

  - No pueden contener espacios.
  - No pueden comenzar con “xml” (en ninguna combinación de mayúsculas y minúsculas).

- No pueden comenzar con ":".

Ciertos caracteres especiales deben reemplazarse con entidades XML:

- < → <
- → >
- & → &
- " → "
- ' → '

- Además, el contenido de los elementos no puede incluir la cadena "]]>", ya que esto es incompatible con SGML y puede causar problemas de interpretación. También, ciertos caracteres como <, >, &, " y ' deben ser reemplazados por sus correspondientes entidades XML para evitar problemas de interpretación.

Tabulador	/t		TAB
Nueva línea	/n
LF
Retorno de carro	/r
CR
Espacio	/s		NO-BREAK SPACE

- Los nombres de las etiquetas deben ser autodescriptivos para facilitar su uso.
- Los atributos deben ir siempre entre comillas.
- Los documentos XML pueden incluir comentarios, los cuales no son interpretados por el procesador XML. Estos comentarios se delimitan con la cadena "<!--" al principio y "-->" al final, y no se permiten dentro de una etiqueta o antes del prólogo.
- No pueden ir dentro de etiquetas ni antes del prólogo del documento.
- Todos los documentos XML deben seguir las reglas sintácticas definidas en la recomendación del estándar XML de W3C.

En conclusión, un documento XML estará bien formado si respeta estas reglas de sintaxis.

Será válido si además cuenta con una declaración de tipos (DTD o XSD) y cumple las reglas definidas en dicha declaración.

6.1 Definición de tipo de documento (DTD, Document Type Definition)

Definición

Las Definiciones de Tipo de Documento, DTD, son utilizadas para definir la estructura de un documento XML, estableciendo los elementos permitidos y su ubicación, así como también su contenido y atributos.

El documento XML debe **cumplir las restricciones** que le haya impuesto en el DTD para ser válido.

6.1.1 Inconvenientes de DTD

Los principales inconvenientes de DTD son:

- Su sintaxis no es XML, lo que dificulta su integración con otras tecnologías modernas.
- Carecen de soporte para espacios de nombres (namespaces).
- No permiten definir tipos de datos complejos, solo admiten datos textuales en los elementos terminales.
- No es posible trabajar con secuencias no ordenadas, ni crear claves combinadas a partir de varios atributos o elementos.
- Una vez definido un DTD, no se pueden añadir nuevos vocabularios.
- La instrucción <!DOCTYPE> es la que indica qué DTD se usará para validar el documento.
  - Debe colocarse al inicio del documento XML.
  - El primer dato especificado es el nombre del elemento raíz del documento.
  - Su sintaxis varía dependiendo del tipo de DTD utilizado.

6.1.2 Declaración

Las características que definen el tipo son:

- **Ubicación de las reglas**
  - Interno: Las reglas aparecen en el propio documento XML.

- Externo: Las reglas aparecen en un archivo independiente.
- Mixto: Mezcla de los anteriores, las reglas aparecen en ambos lugares. Las reglas internas tienen prioridad sobre las externas.

- **Carácter**: si es un DTD para uso privado o público.
  - Para uso privado: Se identifica por la palabra SYSTEM.
  - Para uso público: Se identifica por la palabra PUBLIC. Debe ir acompañado del FPI (Formal Public Identifier - Identificador Público Formal), una etiqueta que identifica al DTD de manera "universal".

Las distintas combinaciones son:

Sintaxis	Tipo de DTD
<!DOCTYPE elemento-raíz [reglas)>	Interno (privado)
<!DOCTYPE elemento-raíz SYSTEM URL>	Externo y privado
<!DOCTYPE elemento-raíz SYSTEM URL [reglas)>	Mixto y privado
<!DOCTYPE elemento-raíz PUBLIC FPI URL>	Externo y público
<!DOCTYPE elemento-raíz PUBLIC FPI URL [reglas]>	Mixto y público

6.1.3 Componentes

Hay cuatro tipos posibles de componentes que se pueden declarar en un DTD:

- **Elemento**: Es una **declaración de tipo de elemento**. Indica la **existencia de un elemento** en el documento XML.

  Sintaxis: `<!ELEMENT nombre_elemento  modelo_contenido>`

  Ejemplo

```
<!ELEMENT titulo (#PCDATA)>
```

  En este caso, se define que el elemento <titulo> puede contener únicamente texto.

- **Atributo**: Es una **declaración de tipo de atributo**. Indica **la existencia de atributos de un elemento** en el documento XML. En general se utiliza un solo ATTLIST para declarar todos los atributos de un elemento (aunque se podría usar un ATTLIST para cada atributo).

  Sintaxis general:

  ```
  <!ATTLIST nombre_elemento
                  nombre atributo tipo_atributo caracter
                  nombre atributo tipo_atributo caracter
  >
  ```

  Ejemplo

```
<!ATTLIST libro
   id ID #REQUIRED
   categoria CDATA #IMPLIED
>
```

Aquí se indica que el elemento <libro> debe tener obligatoriamente un atributo id (único dentro del documento) y opcionalmente un atributo categoria.

- **Entidad**: Referencia a entidades generales, se utilizarán dentro del documento XML.

  Existen distintos tipos:

  - Entidad general (interna):

```
<!ENTITY nombre_entidad "definicion_entidad">
```

  - Entidad general (externa):

```
<!ENTITY nombre_entidad SYSTEM "url_archivo">
```

  - Entidad de parámetro (interna, para usar dentro del propio DTD):

```
<!ENTITY % nombre_entidad "definicion_entidad">
```

  - Entidad de parámetro externa (archivo independiente):

```
<!ENTITY % nombre_entidad PUBLIC "fpi" "url_archivo">
```

- Ejemplo de uso dentro de un documento:

```
<!ENTITY autor "Gabriel García Márquez">
<libro>
  <titulo>Cien años de soledad</titulo>
  <autor>&autor;</autor>
</libro>
```

La notación se utiliza para describir formatos de datos no XML, como imágenes o tipos MIME.

Se declara en el DTD y puede asignarse a atributos de un elemento.

Sintaxis general de la notación:

```
<!NOTATION nombre_notacion SYSTEM "identificador_externo">
```

Sintaxis general del atributo que la usa:

```
<!ATTLIST nombre_elemento nombre atributo NOTATION valor defecto>
```

Ejemplo

```
<!NOTATION gif SYSTEM "image/gif">
<!ATTLIST imagen formato NOTATION (gif) #REQUIRED>
```

Más info

Para ampliar información sobre DTDs puedes hacer clic en el siguiente enlace:

https://www.mclibre.org/consultar/xml/lecciones/xml-dtd.html

6.2 Esquema XML (XSD, XML Schema Definition)

Definición

Un esquema XML es un **mecanismo para comprobar la validez de un documento XML**. Se trata de una **forma alternativa a los DTD**, pero con ciertas ventajas sobre éstos

Ventajas:

- Existe un extenso catálogo de tipos de datos predefinidos para elementos y atributos que pueden ser ampliados o restringidos para crear nuevos tipos.
- Las restricciones de contenido permiten especificar con exactitud la cardinalidad de un elemento, es decir, la cantidad de veces que puede aparecer en un documento XML.
- La característica principal de los espacios de nombres en XML es que permiten mezclar distintos vocabularios o conjuntos de etiquetas en un mismo documento.

Desventaja:

- Los esquemas XML son más difíciles de interpretar por el ojo humano que los DTD.

En un **esquema XML se describe lo que un documento XML puede contener:** qué elementos, con qué atributos, de qué tipos de datos son tanto elementos como atributos, en qué orden aparecen los elementos, cuántas ocurrencias puede haber de cada elemento, etc.

Sabías que

Se pueden definir esquemas muy restrictivos o poco restrictivos. Los primeros fuerzan a los documentos XML validados a ser más precisos.

El XSD está basado en XML y, por tanto, debe cumplir con las reglas de XML:

- Aunque no es obligatorio, normalmente siempre se empieza el fichero con la declaración XML.
- Sólo hay un elemento raíz, que en este caso es <schema>.

El archivo de referencia para definir elementos y atributos que pueden aparecer en un documento XML válido es <xs:schema>. Los elementos hijos inmediatos de este ejemplar se definen con la etiqueta <xs:element> y con atributos se especifica, como mínimo, el **nombre** y, opcionalmente, el **tipo de datos** que contendrá el elemento.

Ejemplo de esquema básico:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

 <xs:element name="persona">
  <xs:complexType>
   <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="edad" type="xs:integer"/>
   </xs:sequence>
  </xs:complexType>
 </xs:element>

</xs:schema>
```

Este esquema indica que un documento XML válido debe contener un elemento <persona> que tenga obligatoriamente un <nombre> (texto) y una <edad> (número entero).

6.2.1 Asociación con documentos XML

Para vincular un esquema XSD con un documento XML, es necesario declarar un espacio de nombres en el documento XML. Este espacio señala la ubicación del archivo de esquema mediante su URI, utilizando el prefijo xsi.

Además, se debe indicar el espacio de nombres de XSD con el atributo xmlns:xsi.

Ejemplo de asociación:

```
<?xml version="1.0" encoding="UTF-8"?>
<persona xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:noNamespaceSchemaLocation="persona.xsd">

 <nombre>Lucía</nombre>
 <edad>29</edad>

</persona>
```

En este ejemplo, el documento XML está vinculado al esquema persona.xsd. Gracias a esta asociación, el procesador XML podrá validar si el contenido del documento cumple con las reglas establecidas en dicho esquema.

6.2.2 Elementos de lenguaje

Este lenguaje tiene la capacidad de manejar tanto datos simples como estructuras de datos complejas, compuestas por la combinación de otros datos simples o complejos. Aparte de las etiquetas **xs:schema** y **xs:element**, algunos de los elementos más utilizados para crear XSD son:

- Tipos complejos, xs:complexType, que define tipos complejos.
- Tipos simples, xs:simpleType, que permite definir un tipo simple restringiendo sus valores.
- Restricciones, xs:restriction, que establecen restricciones sobre elementos de tipo base.
- Agrupaciones, xs:group, que nombran grupos de elementos y atributos para referirse a ellos.

Asimismo, otras etiquetas comunes para crear XSD son:

- Secuencias, xs:sequence, que permiten construir elementos complejos mediante la enumeración de sus componentes.
- Elecciones, xs:choice, que representan opciones alternativas, teniendo en cuenta que son o-exclusivas.
- Contenido mixto, definido mediante el atributo mixed del elemento xs:complexType con valor true, que permite la combinación de texto con elementos.

- Secuencias no ordenadas, xs:all, que representan todos los elementos en cualquier orden.

Ejemplo de uso combinado:

```
<xs:element name="producto">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="nombre" type="xs:string"/>
      <xs:element name="precio" type="xs:decimal"/>
      <xs:choice>
        <xs:element name="digital" type="xs:boolean"/>
        <xs:element name="fisico" type="xs:string"/>
      </xs:choice>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

En este caso, el esquema define que un <producto> debe tener obligatoriamente un <nombre>, un <precio> y elegir entre un valor booleano <digital> o una descripción de tipo <fisico>.

6.2.3 Tipos de datos XSD

XML Schema dispone de numerosos tipos de datos predefinidos para los elementos o atributos, siendo los más comunes:

- **String** se corresponde con una cadena de caracteres UNICODE.
- **Boolean** se trata de un tipo de datos que puede representar valores lógicos, es decir, únicamente puede tomar dos valores: true o false.
- **Integer**, número entero positivo o negativo.
- **PositiveInteger**, **negativeInteger** número entero positivo/negativo
- **Decimal**, número decimal, por ejemplo, 8,97.
- **DateTime**: Representa una fecha y hora absolutas en formato (AAAA-MM-DD T HH:MM:SS)
- **Time**: Hora en el formato hh:mm:ss.
- **Date**: Fecha en formato (aaaa-mm-dd).
- **gYearMonth**: Representa un mes de un año determinado mediante el formato CCYY-MM.

- **gMonth**: Representa el mes mediante el formato –MM. Por ejemplo, febrero es –02.
- **AnyURI**: Representa una URI.
- **Language**: Representa los identificadores de lenguaje, sus valores están definidos en RFC 1766.
- **ID, IDREF, ENTITY, NOTATION, MTOKEN**: Representan lo mismo que en los DTD's.

Aparte de los datos proporcionados por el sistema, a veces es interesante crear tipos de datos nuevos personalizados que tienen mayor precisión y eficiencia.

Para ello, XSD divide los elementos en dos grandes grupos basándose en los datos que contienen:

- **Elementos con contenido de tipo simple**: Son elementos sin atributos que sólo contienen datos.
- **Elementos con contenido de tipo complejo**: Son elementos que pueden tener atributos, no tener contenido o contener elementos.

Ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<libro xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:noNamespaceSchemaLocation="Ejemplo.xsd">
   <título>XML práctico</título>
   <autor>Sébastien Lecomte</autor>
   <autor>Andrew Watt</autor>
   <editorial>Ediciones ENI</editorial>
</libro>
```

Código fuente documento XML

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name="libro" type="Tlibro"/>
   <xs:complexType name="Tlibro"> <!- Contiene otros
elementos -->
      <xs:sequence>
         <xs:element name="título" type="xs:string"/>
         <xs:element name="autor" type="xs:string"/>
         <xs:element name="autor" type="xs:string"/>
         <xs:element name="editorial" type="xs:string"/>
      </xs:sequence>
```

```
    </xs:complexType>
</xs:schema>
```

Código fuente documento XSD

6.2.4 Facetas de los tipos de datos

En los esquemas XML podemos aplicar restricciones sobre los valores de los datos de un elemento o atributo.

Las facetas son definiciones que solo se aplican a tipos simples utilizando el elemento xs:restriction. Se refieren a elementos que se utilizan para restringir el contenido de otro elemento en un documento XML. Estos elementos pueden combinarse para restringir aún más el valor permitido del elemento en cuestión. Normalmente se representan como elementos secundarios dentro de una restricción. Algunas de las facetas más utilizadas incluyen **length**, **minlength** y **maxlength**, que limitan la longitud del tipo de datos, así como **enumeration**, que restringe el valor del tipo de datos a un conjunto específico de valores.

Ejemplo: Dado el siguiente documento XML llamado Ejemplo1.xml, genera el correspondiente documento XSD para validarlo con las siguientes condiciones: el elemento "estado" debe ser una cadena de texto con una longitud máxima de 9 caracteres y dos valores posibles: "conectado" y "ocupado".

```
<?xml version="1.0" encoding="UTF-8"?>
<usuarios xmlns:xs="http://www.w3.org/2001/XMLSchema-
instance" xs:noNamespaceSchemaLocation="Ejemplo1.xsd">
    <usuario>
        <nombre>Peter Green</nombre>
        <departamento>Finanzas</departamento>
        <puntuación>5</puntuación>
        <estado>conectado</estado>
    </usuario>
</usuarios>
```

Código fuente documento XML

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name="usuarios">
   <xs:complexType>
      <xs:sequence>
         <xs:element name="usuario">
            <xs:complexType>
               <xs:sequence>
                  <xs:element name="nombre"
type="xs:string"/>
                  <xs:element name="departamento"
type="xs:string"/>
                  <xs:element name="puntuación"
type="xs:integer"/>
                  <xs:element name="estado"
type="Testado"/>
               </xs:sequence>
            </xs:complexType>
         </xs:element>
      </xs:sequence> ….

      </xs:sequence>
     </xs:complexType>
</xs:element>
<xs:simpleType name="Testado">
   <xs:restriction base="xs:string">
      <xs:maxLength value="9"/>
      <xs:enumeration value="conectado"/>
      <xs:enumeration value="ocupado"/>
   </xs:restriction>
</xs:simpleType>
</xs:schema>
```

Código fuente documento XSD

Pattern: Permite construir máscaras que han de cumplir los datos de un elemento. A continuación, se presenta una tabla con algunos de los caracteres que poseen un significado especial en la creación de las máscaras.

Patrón	Significado
[A-Z a-z]	Letra.
[A-Z]	Letra mayúscula.
[a-z]	Letra minúscula.
[0-9]	Dígitos decimales.

\D	Cualquier carácter excepto un dígito del 0 al 9.
(A)	Cadena que coincide con A.
A | B	Cadena que es igual a la cadena A o a la B.
AB	Cadena que es la concatenación de las cadenas A y B.
A?	Cero o una vez la cadena A.
A+	Una o más veces la cadena A.
A*	Cero o más veces la cadena A.
[abcd]	Alguno de los caracteres que están entre corchetes.
[^abcd]	Cualquier carácter que no esté entre corchetes.
\t	Tabulador.

Ejemplo: Dado el siguiente documento XML llamado Ejemplo1.xml, genera el correspondiente documento XSD para validarlo con las siguientes condiciones: el elemento "nombre" debe admitir nombres compuestos que comiencen con mayúscula.

```
<?xml version="1.0" encoding="UTF-8"?>
<usuarios xmlns:xs="http://www.w3.org/2001/XMLSchema-
instance" xs:noNamespaceSchemaLocation="Ejemplo1.xsd">
   <usuario>
      <nombre>Peter Green</nombre>
      <departamento>Finanzas</departamento>
      <puntuación>5</puntuación>
      <estado>conectado</estado>
   </usuario>
</usuarios>
```

Código fuente documento XML

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name="usuarios">
   <xs:complexType>
      <xs:sequence>
         <xs:element name="usuario">
            <xs:complexType>
               <xs:sequence>
                  <xs:element name="nombre"
type=”Tnombre"/>
                  <xs:element name="departamento"
type="xs:string"/>
                  <xs:element name="puntuación"
type="xs:integer"/>
                  <xs:element name="estado"
type="Testado"/>
               </xs:sequence>
            </xs:complexType>
         </xs:element>
        </xs:sequence>

        </xs:sequence>
     </xs:complexType>
</xs:element>
<xs:simpleType name="Testado">
   <xs:restriction base="xs:string">
      <xs:maxLength value="9"/>
      <xs:enumeration value="conectado"/>
      <xs:enumeration value="ocupado"/>
   </xs:restriction>
</xs:simpleType>
<xs:simpleType name="Tnombre">
   <xs:restriction base="xs:string">
      <xs:pattern value="(([A-Z]+[a-zñáéíóú]*)+[\s]*)*"/>
   </xs:restriction>
</xs:simpleType>
</xs:schema>
```

Código fuente documento XSD

7. Utilización de espacios de nombre en XML

Se trata de un mecanismo de resolución de conflictos de nombres en documentos XML que permite distinguir elementos o atributos con nombres idénticos, pero definiciones diferentes. Esto se logra mediante la asignación de un **identificador único** (conocido como **URI**) a cada espacio de nombres utilizado en el documento. De esta forma se puede diferenciar entre elementos o atributos que tengan el mismo nombre, pero que pertenezcan a diferentes espacios de nombres, y evitar confusiones o errores en la interpretación del documento.

Para declarar un espacio de nombres en un documento XML se utiliza un atributo especial llamado "**xmlns**". Este atributo se agrega a los elementos que se desean incluir en el espacio de nombres y se define de la siguiente manera:

```
<nombre_elemento xmlns:prefijo="URI_del_espacio_de_nombres">
```

Aquí, "nombre_elemento" es el nombre del elemento al que se le está agregando el atributo, "prefijo" es el prefijo que se le da al espacio de nombres y "URI_del_espacio_de_nombres" es la identificación única del espacio de nombres.

Una vez que se ha declarado el espacio de nombres, se puede hacer referencia a los elementos y atributos que pertenecen a él utilizando el prefijo asociado y el carácter dos puntos ":". Esto permite diferenciar entre elementos y atributos con el mismo nombre, pero con definiciones diferentes dentro de un mismo documento XML.

Ejemplo:

```
<info:pedido xmlns:info="empresa:espacios:info">
   <info:item info:id="010">Batidora eléctrica</info:item>
   .....
</info:pedido>
```

Código fuente utilización de espacios de nombres

En este caso:

- info es el prefijo asignado.
- "empresa:espacios:info" es el URI que identifica el espacio de nombres.
- Los elementos <info:pedido> y <info:item> quedan dentro de ese espacio.

Cualquier cadena de texto puede ser usada como prefijo del espacio de nombres. Una de las reglas para los espacios de nombres en XML es que su URI debe ser único, aunque en realidad no se verifica mediante ninguna conexión. El URI no es más que un nombre lógico del espacio de nombres. A veces se usa como URI http://-

Más info

Para ampliar información sobre URI puedes hacer clic en el siguiente enlace: URL o URI

Una vez declarado un espacio de nombres, este puede ser utilizado en diferentes lugares de un documento XML, es decir, su ámbito de uso abarca los elementos donde se hace referencia a él y sus elementos hijos. Para referirse a un elemento utilizando su espacio de nombres, se debe anteponer un prefijo a su nombre.

Se debe tener en cuenta que un elemento descendiente de otro tiene la capacidad de declarar un espacio de nombres diferente al de su elemento padre. Esto brinda la posibilidad de que cada elemento tenga su propio espacio de nombres, permitiendo diferenciar entre elementos con el mismo nombre, pero pertenecientes a diferentes vocabularios.

Ejemplo: En un documento XML se declaran dos espacios de nombres, empresa:espacios:dept y empresa:espacios:emp, de prefijos dept y emp respectivamente.

```
<dept:departamentos xmlns:dept="empresa:espacios:dept"
xmlns:emp="empresa:espacios:emp">
   <dept:departamento dept:deptno= "5">
      <emp:empleado emp:empno="1200">
         <emp:nombre>Roberto</emp:nombre>
         <emp:apellido>Castro</emp:apellido>
      </emp:empleado>
      <emp:empleado emp:empno="1201">
         <emp:nombre>María</emp:nombre>
         <emp:apellido>Sánchez</emp:apellido>
      </emp:empleado>
   </dept:departamento>
</dept:departamentos>
```

Código fuente utilización de espacios de nombres

```
▼<dept:departamentos xmlns:dept="empresa:espacios:dept" xmlns:emp="empresa:espacios:emp">
  ▼<dept:departamento dept:deptno="5">
    ▼<emp:empleado emp:empno="1200">
       <emp:nombre>Roberto</emp:nombre>
       <emp:apellido>Castro</emp:apellido>
     </emp:empleado>
    ▼<emp:empleado emp:empno="1201">
       <emp:nombre>María</emp:nombre>
       <emp:apellido>Sánchez</emp:apellido>
     </emp:empleado>
   </dept:departamento>
 </dept:departamentos>
```

Visualización resultante código fuente de espacios de nombres

Los atributos pueden pertenecer al mismo espacio de nombres al que pertenece el elemento en el que están asociados o a otro diferente.

La definición es la misma que con los elementos, anteponiendo el prefijo del espacio de nombres y los dos puntos al nombre el atributo.

Si no aparece ningún prefijo de espacio de nombres, el atributo no pertenecerá a ningún espacio de nombres.

Ejemplo:

El atributo trabajo:empno del elemento <emp:datosPersonales> pertenece a un espacio de nombres diferente que este.

El prefijo del espacio de nombres del atributo es trabajo. El atributo deptno del elemento <emp:departamento> no pertenece a ningún espacio de nombres.

Dos atributos con el mismo nombre pero con distinto prefijo de espacio de nombres son diferentes, lo que significa que pueden estar asociados al mismo elemento.

Ejemplo:

Los atributos trabajo:empno y emp:empno son diferentes, luego pueden aparecer asociados al mismo elemento.

En XML un espacio de nombres por defecto es aquel en el que no se ha definido un prefijo. Los espacios de nombres se utilizan para evitar conflictos en la identificación de elementos o atributos que tengan el mismo nombre en diferentes vocabularios. El espacio de nombres en XML tiene un ámbito de aplicación limitado. Se aplica al elemento donde se declara y a todos sus elementos descendientes, pero no a sus atributos. Esto significa que, al declarar un espacio de nombres en un elemento, este solo se aplica a las etiquetas de ese elemento y las etiquetas de los elementos anidados.

Los atributos dentro de esas etiquetas no están incluidos en el espacio de nombres y se tratan como atributos regulares.

Es importante tener en cuenta esto al trabajar con XML y definir espacios de nombres para evitar confusiones o errores en el procesamiento de datos.

Si se tuviera que añadir un espacio de nombres con prefijo a un documento XML grande ya creado, habría que ir añadiendo el prefijo a los elementos y atributos, algo tedioso y tendente a producir errores. De esta manera, se puede evitar tener que escribir el prefijo.

El mayor inconveniente de esta medida, citado ya antes, es que el espacio de nombres por defecto sólo afecta al elemento en el que está declarado y a sus descendientes, no a los atributos.

Ejemplo:

El espacio de nombres por defecto, empresa:espacios:emp, no afectará al atributo empno del elemento <datosPersonales> ni al atributo deptno del elemento <departamento>.

Se puede desasignar a un elemento de un espacio de nombres por defecto con la orden: <nombre elemento xmlns="">

8. Resumen

XML y **JSON** son dos de los lenguajes más utilizados para el almacenamiento y transmisión de información en aplicaciones web y móviles. **XML** es un lenguaje de marcado extensible que tiene una sintaxis relativamente sencilla y que permite definir etiquetas y estructuras personalizadas para representar la información.

Los documentos XML pueden ser validados mediante DTD (Document Type Definition) o XML Schema (XSD), garantizando su integridad y coherencia estructural. Además, los espacios de nombres proporcionan un mecanismo para evitar conflictos cuando se usan etiquetas con el mismo nombre en diferentes contextos, lo que hace posible integrar múltiples vocabularios en un único documento.

Finalmente, tanto XML como JSON cuentan con herramientas de edición y validación, que facilitan la creación, comprobación y mantenimiento de documentos bien formados y válidos en proyectos reales.

Tech Hub <FP> euroformac

Glosario

- **SGML**: SGML es la abreviatura de Lenguaje de Marcado Generalizado Estándar o Standard Generalized Markup Language. Es una norma ISO que se deriva de su predecesor, el GML de IBM. SGML proporciona la capacidad de definir la estructura de un documento en base a la relación lógica entre sus componentes.
- **Namespace**: Un namespace (espacio de nombres) es un contenedor abstracto donde pueden existir uno o más identificadores únicos. Cada identificador definido en un namespace está asociado con ese espacio de nombres.
- **JavaScript**: JavaScript es un lenguaje de programación para implementar funciones complejas en páginas web, que permite mostrar actualizaciones de contenido, mapas interactivos, animaciones de gráficos, reproductores de vídeo y más. Es la tercera capa de las tecnologías web estándar después de HTML y CSS.
- **Array**: Un array, también conocido como arreglo o vector, es una estructura de datos en programación que almacena una colección ordenada de elementos del mismo tipo. Cada elemento del array puede ser accedido mediante un índice que representa su posición dentro del arreglo. Los arrays son utilizados frecuentemente para almacenar y manipular conjuntos de datos de manera eficiente en la memoria de un ordenador. Además, permiten realizar operaciones matemáticas y de búsqueda y ordenamiento de datos de manera sencilla.
- **String**: Un string, también conocido como cadena de caracteres, es una secuencia ordenada y finita de caracteres, incluyendo letras, números, símbolos y espacios en blanco, que se utilizan para representar texto en la programación. En general, un string puede ser considerado como una cadena de caracteres que forman una palabra, una frase o un párrafo. Los strings se utilizan en diferentes contextos, como en la entrada y salida de datos en programas, la manipulación de texto, el almacenamiento de información, la comunicación entre sistemas, entre otros.
- **Null**: En programación, **null** es un valor especial que se utiliza para indicar que una variable o un puntero no apuntan a ningún objeto o dato válido en la memoria. Es decir, **null** representa la ausencia intencional de un valor o una referencia. Se utiliza comúnmente en los lenguajes de programación para inicializar variables o punteros y para indicar que una variable o un objeto no tienen ningún valor o contenido. En algunos lenguajes de programación, **null** se puede utilizar como sinónimo de **false, 0** o una cadena vacía.

- **True**: En programación, el valor booleano "true" se utiliza para representar el valor de verdad "verdadero" en la lógica booleana o lógica binaria. En términos simples, esto significa que el valor booleano "true" indica que una condición es verdadera o que una expresión es cierta. En muchos lenguajes de programación, el valor booleano "true" se representa con el número 1.

- **False**: El valor "false" es un valor booleano que se corresponde con el valor de "falso" en la lógica clásica. En programación, se utiliza para representar una condición que no se cumple o un valor que no es verdadero. Es decir, se utiliza para indicar que algo es falso o incorrecto.

- **REST**: REST (Representational State Transfer) es un estilo arquitectónico para la creación de servicios web que se basa en el uso de los protocolos y estándares de la Web. La idea principal detrás de REST es que los recursos (por ejemplo, documentos, imágenes, etc.) son identificados y manipulados a través de sus identificadores únicos (URI), y que los servicios REST deben ser stateless, es decir, que no deben almacenar información sobre el estado de las solicitudes anteriores. REST se basa en HTTP y se utiliza ampliamente para la creación de APIs (Application Programming Interfaces) para aplicaciones web y móviles.

- **XSLT**: XSLT (Extensible Stylesheet Language Transformations) es un lenguaje de transformación de documentos XML, que permite definir una hoja de estilo XML para transformar documentos XML en otros formatos, tales como HTML, XHTML, XML, PDF, entre otros. XSLT utiliza reglas de transformación para definir cómo los elementos y atributos del documento fuente deben ser transformados en el documento resultante. Es una tecnología estándar del W3C y es ampliamente utilizada en aplicaciones de publicación de contenido y conversión de formatos de datos.

- **FPI**: Un Formal Public Identifier (FPI) es una cadena de texto utilizada para identificar un producto, especificación o documento de manera unívoca y universalmente reconocida. Los FPI son utilizados en varias tecnologías de la web, como HTML, XML y SGML, y son especialmente útiles para garantizar la interoperabilidad y portabilidad de los documentos en diferentes sistemas y plataformas. Un FPI está compuesto por una serie de campos que incluyen información sobre el productor, la versión y el formato del documento.

Bibliografía

- Zurdo, S. F. J. T. R. & RA-MA Editorial. (2023). LENGUAJES DE MARCAS Y SISTEMAS DE GESTIÓN DE LA INFORMACIÓN. CFGS. (1.a ed.). RA-MA Editorial.
- Castro, J. M., & Rodríguez, J. R. (2001). Lenguajes de marcas y sistemas de gestión de la información. Garceta.
- Mercier, C., Bird, A. J., & Swick, R. (s. f.). World Wide Web Consortium (W3C). https://www.w3.org/
- International Organization for Standardization. (2023, 15 marzo). ISO. https://www.iso.org/home.html
- Apuntes de Lenguajes de marcas y sistemas de gestión de información – DAW. (s. f.). https://javiergutierrez.trade/apuntes-de-lenguajes-de-marcas-y-sistemas-de-gestion-de-informacion-daw/
- Morales, R. (2021, 11 junio). Lenguajes de Marcas y Sistemas de Gestión de Información. https://www.ticarte.com/contenido/lenguajes-de-marcas-y-sistemas-de-gestion-de-informacion